CSC 8607 – Introduction au Deep Learning

CI4 : Réseaux récurrents et génération de séquences

Génération de partitions musicales au format ABC

Maroua Zoubir
Télécom SudParis – MAIA

1er novembre 2026

1 Exploration et prétraitement des partitions ABC

1.1 Chargement des données

Le script `train_music.py` charge les fichiers `train.json` et `validation.json` situés dans `/mnt/ssd/irishman`. Le jeu d'entraînement contient **214 122** partitions et le jeu de validation **2 162** partitions.

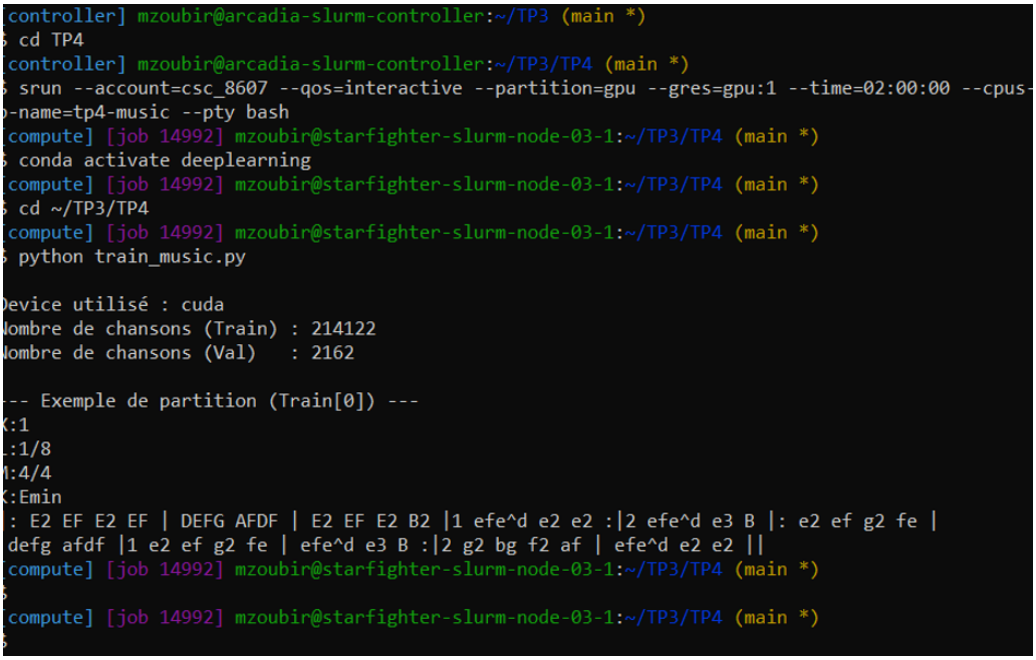


Figure 1 – Chargement des données et affichage de la première partition (`Train[0]`).

La première partition est la suivante :

```
X:1
L:1/8
M:4/4
K:Emin
|: E2 EF E2 EF | DEFG AFDF | E2 EF E2 B2 |1 efe^d e2 e2 :|2 efe^d e3 B |: e2 ef g2 fe |
 defg afdf |1 e2 ef g2 fe | efe^d e3 B :|2 g2 bg f2 af | efe^d e2 e2 ||
```



Figure 2 – Rendu de la première partition dans ABC Player.

1.2 Rôle des métadonnées d'en-tête

La notation ABC est un format textuel décrivant une composition musicale. Les lignes d'en-tête précèdent les notes :

- `X:1` : identifiant (numéro) de la partition ;
- `L:1/8` : durée par défaut d'une note, ici la croche ;
- `M:4/4` : métrique, c'est-à-dire une mesure à quatre temps ;
- `K:Emin` : tonalité, ici Mi mineur.

Ces informations permettent au lecteur ABC d'interpréter correctement les lettres (`A`–`G`, `a`–`g`), leurs durées (les chiffres) et leurs altérations. Les barres verticales séparent les mesures, et les symboles de répétition (`|:`, `:|`, `|1`, `|2`) indiquent les passages à rejouer.

1.3 Vocabulaire et padding

Nous travaillons au niveau caractère (*character-level language modeling*). Le vocabulaire est construit à partir de tous les caractères des jeux d'entraînement et de validation, triés avec `sorted(set(all_text))`.

Grandeur	Valeur
Partitions d'entraînement	214 122
Partitions de validation	2 162
Taille du vocabulaire (`vocab_size`)	95
Longueur moyenne d'une partition	290,1 caractères
Longueur maximale	2 968 caractères
`SEQ_LENGTH` retenue	500

Table 1 – Statistiques du jeu de données.

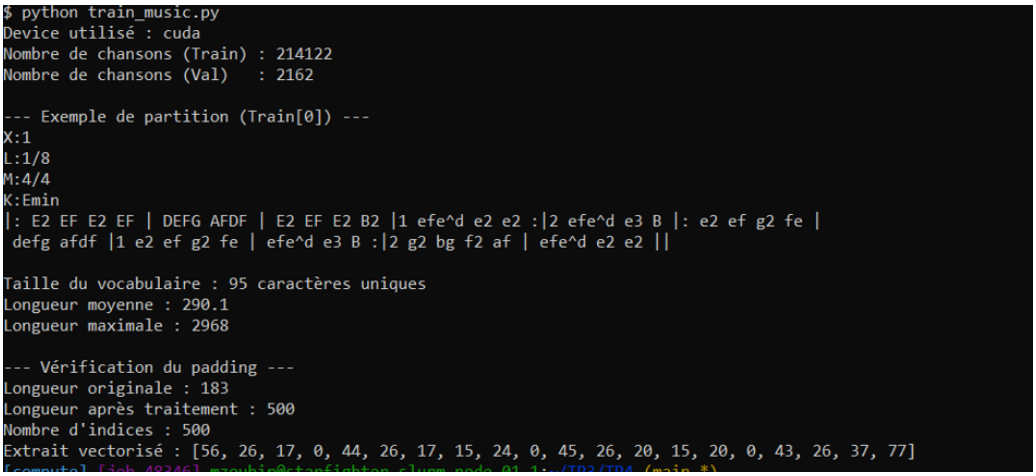


Figure 3 – Statistiques du vocabulaire et vérification du padding (longueur 183 → 500).

Chaque caractère est associé à un indice par les dictionnaires `char_to_index` et `index_to_char`. Les partitions plus courtes que 500 caractères sont complétées par des espaces, les plus longues sont tronquées.

Pourquoi utiliser `sorted()` ? Un `set` Python n'a pas d'ordre garanti : l'ordre d'itération peut varier d'une exécution à l'autre (notamment pour les chaînes, dont le hachage est randomisé). Sans tri, le même indice pourrait désigner un caractère différent entre l'entraînement et la génération. Si l'on sauvegardait les poids aujourd'hui et que l'on relançait le script demain sans `sorted()`, le mapping serait différent : la matrice d'*embedding* et la couche de sortie seraient interprétées avec de mauvais indices, et le modèle produirait un texte incohérent alors que ses poids sont corrects. Le tri garantit un mapping déterministe et reproductible.

Pourquoi tronquer à 500 plutôt qu'à la longueur maximale ? La longueur 500 est un compromis entre conservation de l'information musicale et coût de calcul. Aligner toutes les chansons sur 2 968 caractères aurait pour conséquences :

- **temps de calcul** : la BPTT parcourt des séquences près de six fois plus longues, et la majorité du calcul porterait sur du padding (la longueur moyenne n'est que de 290) ;
- **mémoire GPU** : les activations stockées pour la rétropropagation croissent linéairement avec la longueur, ce qui impose de réduire le `batch_size` ;
- **apprentissage** : le modèle verrait une très forte proportion de caractères de remplissage, ce qui dilue le signal utile, et la rétropropagation sur de très longues séquences aggrave l'instabilité des gradients.

Le prix à payer est la perte éventuelle de la fin des partitions les plus longues.

2 Dataset auto-régressif et modélisation

2.1 Dimensions des tenseurs

Le décalage d'un cran apprend au modèle à prédire le caractère suivant. Chaque exemple a donc 499 éléments et, pour `BATCH_SIZE` = 128 :

$$\boxed{x.\text{shape} = (128,\ 499)} \qquad \boxed{y.\text{shape} = (128,\ 499)}$$

Pourquoi `torch.long` ? Les indices sont des identifiants de catégories (entiers entre 0 et 94), pas des valeurs continues. `nn.Embedding` accède à sa table de vecteurs par indexation entière, et les cibles de `nn.CrossEntropyLoss` doivent être des indices de classes de type `torch.long`. Un `float32` ne permettrait pas d'indexer l'*embedding*.

2.2 Calcul théorique du nombre de paramètres

On prend $V = 95$, $D = 256$ (`embedding_dim`) et $H = 512$ (`hidden_size`).

Embedding. $N_{\text{emb}} = V \times D = 95 \times 256 = \boxed{24\,320}$ (identique pour RNN et LSTM).

RNN. `nn.RNN` possède $W_{ih} \in \mathbb{R}^{H \times D}$, $W_{hh} \in \mathbb{R}^{H \times H}$ et deux biais $b_{ih}, b_{hh} \in \mathbb{R}^{H}$:

$$N_{\text{RNN}} = HD + H^2 + 2H = 131\,072 + 262\,144 + 1\,024 = \boxed{394\,240}.$$

LSTM. Le LSTM effectue quatre transformations (porte d'oubli, porte d'entrée, porte de sortie, candidat de cellule), chacune avec ses propres W_{ih}, W_{hh}, b_{ih} et b_{hh} :

$$N_{\text{LSTM}} = 4\,(HD + H^2 + 2H) = 4 \times 394\,240 = \boxed{1\,576\,960}.$$

Couche linéaire. $N_{\text{fc}} = HV + V = 512 \times 95 + 95 = \boxed{48\,735}$.

Couche	RNN	LSTM
Embedding	24 320	24 320
RNN / LSTM	394 240	1 576 960
Linear (`fc`)	48 735	48 735
Total	**467 295**	**1 650 015**

Table 2 – Nombre de paramètres par couche (valeurs théoriques).

La fonction `count_parameters` confirme ces valeurs : `RNN : 467 295` et `LSTM : 1 650 015` (figure 4).

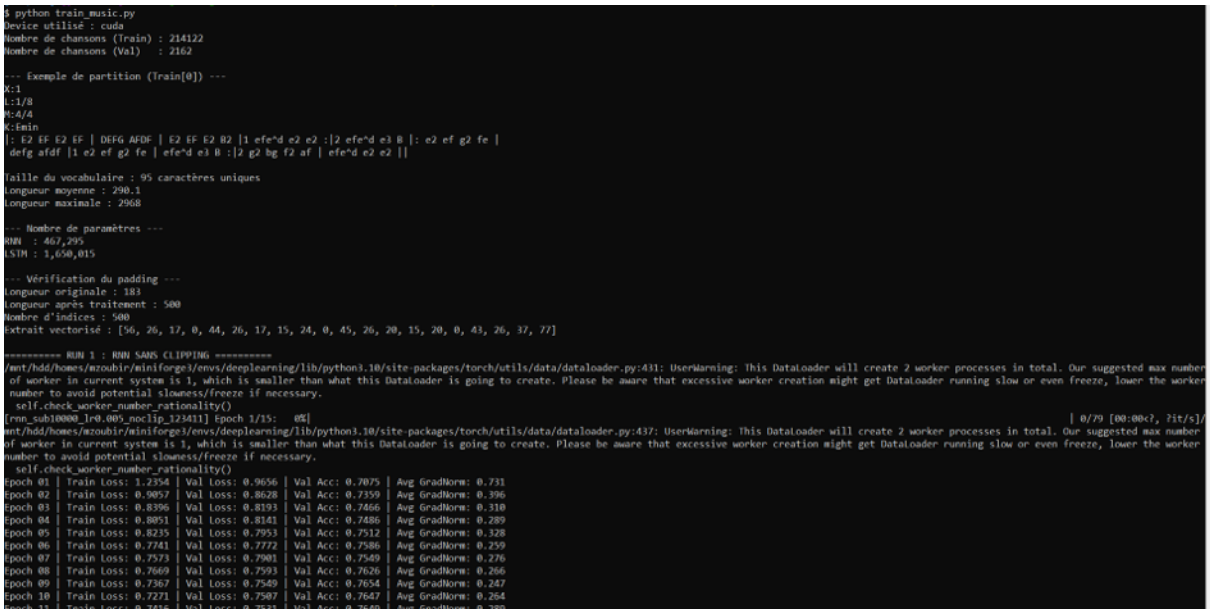


Figure 4 – Sortie du script : nombre de paramètres des deux modèles et début du Run 1.

Pourquoi 4 fois plus de paramètres dans le bloc LSTM ? Le RNN d'Elman effectue une seule transformation récurrente par pas de temps, alors que le LSTM en effectue quatre (trois portes et un candidat de mémoire). Chacune possède ses propres matrices et biais, d'où un facteur exactement 4 sur le *bloc récurrent*. Le modèle complet n'a en revanche pas 4 fois plus de paramètres, car l'*embedding* et la couche linéaire sont identiques ($1\,650\,015/467\,295 \approx 3{,}5$).

2.3 Embedding plutôt que couche linéaire

Les caractères sont des catégories : les indices (A→0, B→1, C→2…) ne sont que des identifiants et n'ont aucune relation d'ordre ou de distance. Passer directement l'indice scalaire dans `nn.Linear(1, embedding_dim)` imposerait une relation linéaire artificielle (le vecteur du caractère 2 serait une fonction affine de celui du caractère 1, « C est deux fois A »). À l'inverse, `nn.Embedding(vocab_size, embedding_dim)` est une table de correspondance : chaque caractère dispose de son propre vecteur de dimension 256, appris librement pendant l'entraînement, sans contrainte numérique entre caractères.

2.4 Structure de `hidden_state`

— `nn.RNN` : `hidden_state` $= h_n$, un tenseur unique de forme (1, 128, 512).

— `nn.LSTM` : `hidden_state` $= (h_n, c_n)$, un tuple de deux tenseurs de forme (1, 128, 512).

h_n est l'état caché final, à partir duquel sont produites les sorties. c_n est l'état de la cellule (mémoire interne) : il est mis à jour de façon additive et contrôlé par les portes, ce qui permet de conserver l'information sur de longues séquences et de limiter la disparition du gradient.

Élément	RNN	LSTM
État retourné	h_n	(h_n, c_n)
Transformations récurrentes	1	4
Paramètres du bloc récurrent	394 240	1 576 960
Mémoire interne dédiée	non	oui

Table 3 – Synthèse RNN vs LSTM.

3 Entraînement, dynamique des gradients et comparaison RNN/LSTM

3.1 Protocole expérimental

Trois entraînements ont été lancés avec 15 époques, `subset_size` = 10 000, `batch_size` = 128, Adam avec lr $= 5 \cdot 10^{-3}$, `embedding_dim` = 256 et `hidden_size` = 512 :

— **Run 1** : RNN sans gradient clipping ;

— **Run 2** : RNN avec gradient clipping (`max_grad_norm` = 1,0) ;

— **Run 3** : LSTM avec gradient clipping (`max_grad_norm` = 1,0).

3.2 Questions sur le code

Pourquoi `logits.reshape(-1, vocab_size)` et `y.reshape(-1)` ? `nn.CrossEntropyLoss` attend des logits de forme (N, C) et des cibles de forme $(N,)$. Or le modèle renvoie des logits de forme (batch, seq, vocab) et les cibles ont la forme (batch, seq). On fusionne donc les dimensions batch et temps : chaque caractère de chaque séquence devient un exemple de classification indépendant parmi $C = 95$ classes. La valeur obtenue à la place du -1 est

$$N = \text{batch_size} \times (\text{SEQ_LENGTH} - 1) = 128 \times 499 = 63\,872$$

pour un batch complet (le dernier batch peut être plus petit).

Pourquoi `clip_grad_norm_` entre `loss.backward()` et `optimizer.step()` ? Les gradients n'existent qu'après `loss.backward()` : clipper avant n'aurait aucun effet (les gradients sont absents ou périmés). Le clipping doit aussi précéder `optimizer.step()`, car c'est cette étape qui consomme les gradients pour mettre à jour les poids : clipper après, c'est modifier des gradients qui ne serviront plus (ils seront remis à zéro au tour suivant). Le clipping redimensionne les gradients de sorte que leur norme globale soit au plus `max_norm`, *avant* qu'ils ne soient utilisés.

3.3 Résultats

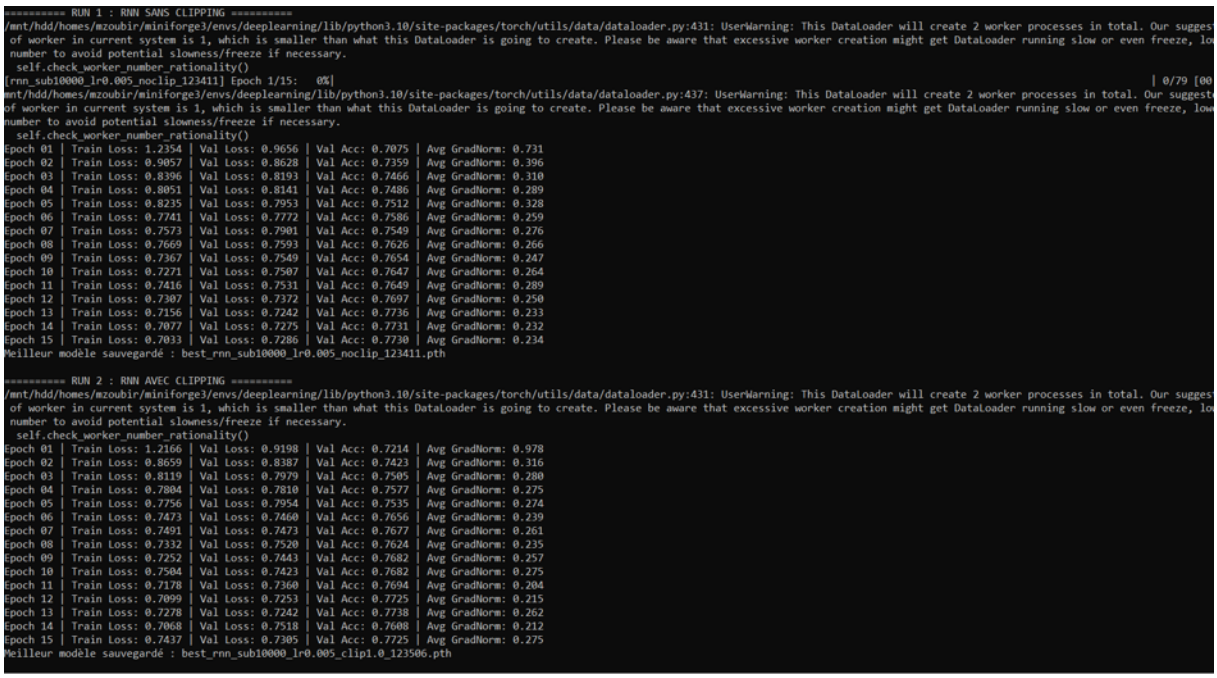


Figure 5 – Logs des Run 1 et Run 2 (RNN sans et avec clipping).

Critère (époque 15)	RNN sans clip.	RNN avec clip.	LSTM avec clip.
Accuracy de validation	77,30 %	77,25 %	**81,57 %**
Loss de validation	0,7286	0,7305	**0,5712**
Loss d'entraînement	0,7033	0,7437	0,5326
GradNorm moyenne	0,234	0,275	0,115
Temps d'entraînement	41,5 s	41,7 s	1 min 13 s

Table 4 – Comparaison finale des trois runs (valeurs relevées dans les logs et TensorBoard).

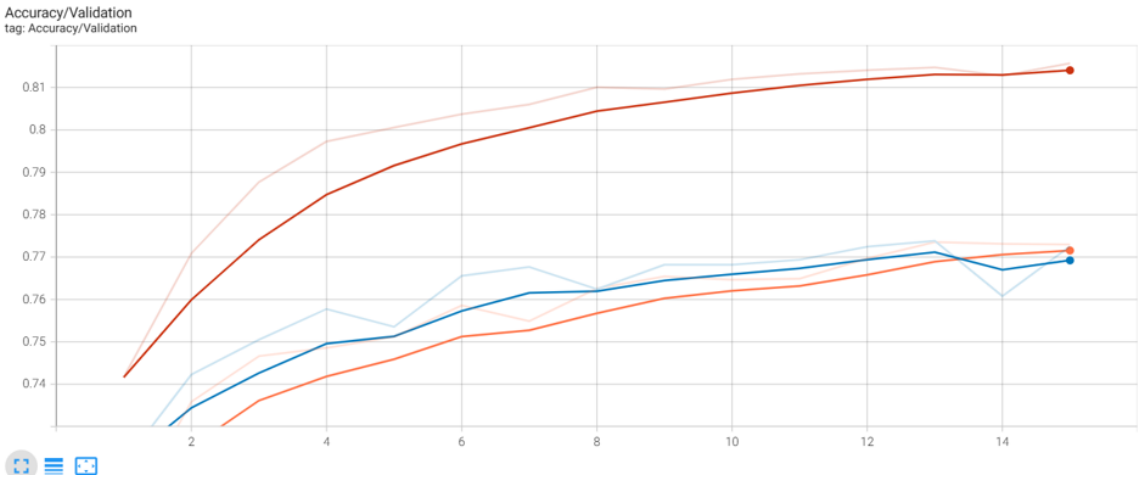


Figure 6 – `Accuracy/Validation`. Rouge : LSTM avec clipping ; bleu : RNN avec clipping ; orange : RNN sans clipping. Les courbes foncées sont lissées, les courbes pâles sont les valeurs brutes.

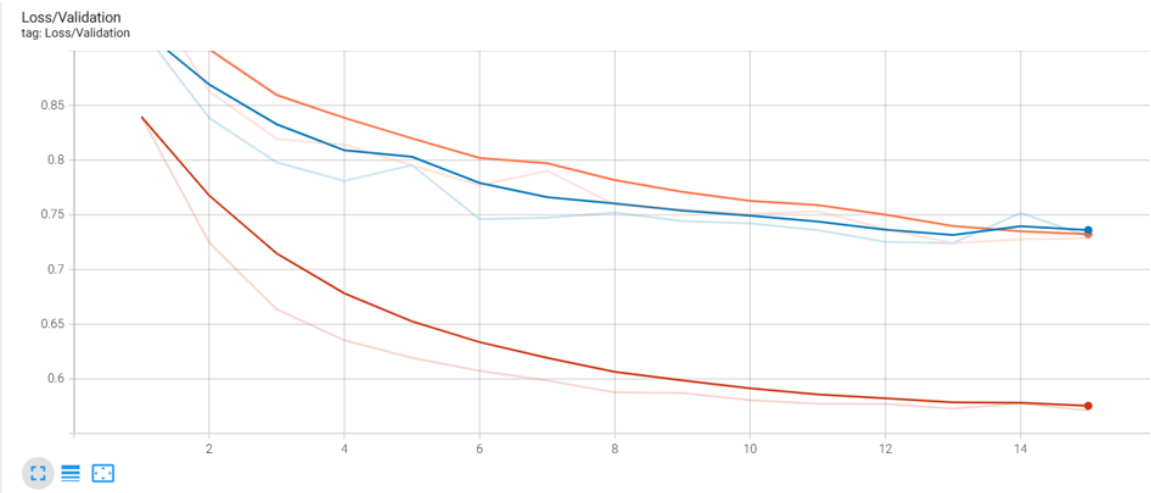


Figure 7 – `Loss/Validation` (mêmes couleurs).

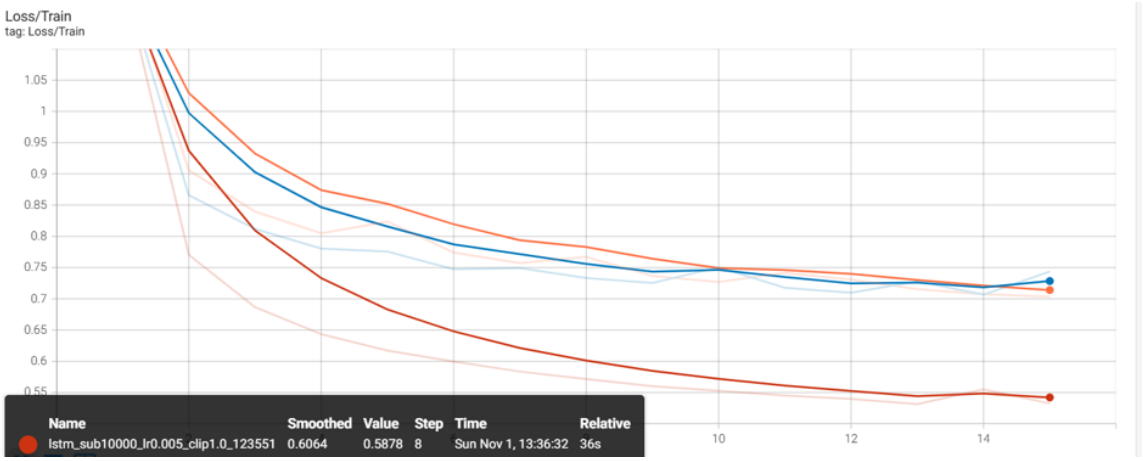


Figure 8 – `Loss/Train` (mêmes couleurs).

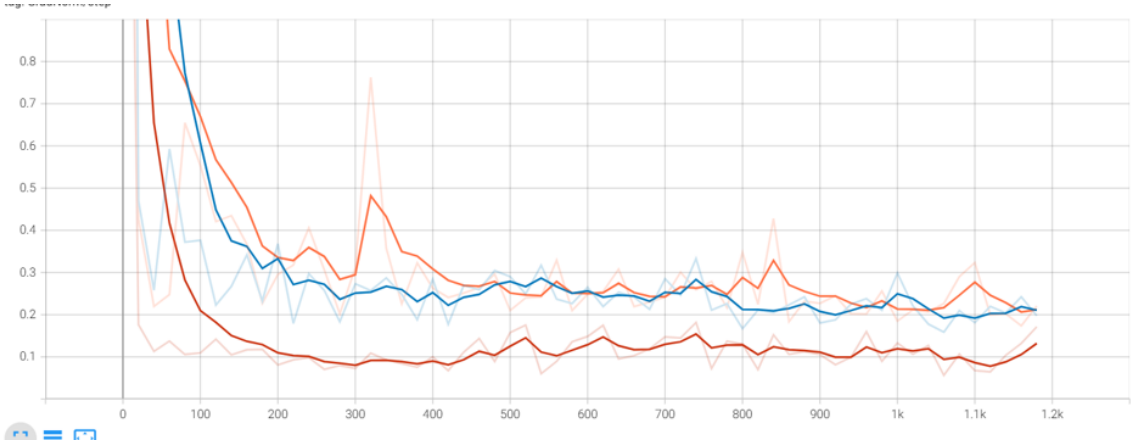


Figure 9 – `GradNorm/step` (mêmes couleurs).

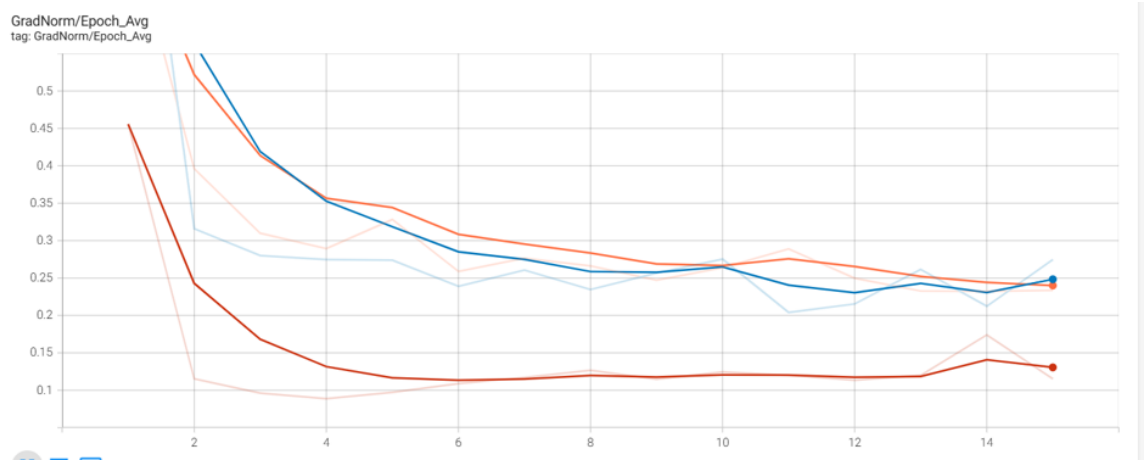


Figure 10 – `GradNorm/Epoch_Avg` (mêmes couleurs).

3.4 Run 1 vs Run 2 : effet du gradient clipping

Dans le code, `clip_grad_norm_` renvoie la norme globale des gradients *avant* clipping : les courbes `GradNorm` montrent donc la norme brute, et non la norme après correction.

Les deux RNN ont un comportement très proche : accuracy de validation de 77,30 % sans clipping et 77,25 % avec clipping, et loss de validation d'environ 0,73 dans les deux cas. Les normes des gradients sont élevées au début (de l'ordre de 0,7 à 1 en moyenne sur la première époque, avec des valeurs instantanées plus fortes), puis se stabilisent autour de 0,2–0,3. On observe quelques pics isolés sur le RNN sans clipping (par exemple vers le pas 300, avec une norme proche de 0,75), mais aucune explosion franche. La courbe de validation du RNN avec clipping présente par ailleurs une irrégularité à l'époque 14 (loss de 0,7518), ce qui montre que le clipping ne garantit pas une courbe plus lisse.

Le gradient clipping cible le problème d'**explosion des gradients** : dans un RNN, la BPTT multiplie le gradient par la même matrice récurrente W_{hh} (et la dérivée de la non-linéarité) à chaque pas de temps. Sur 500 pas, si le rayon spectral de W_{hh} dépasse 1, la norme du gradient croît exponentiellement. En ramenant la norme globale à au plus 1,0, le clipping empêche une mise à jour disproportionnée des poids. Ici, comme la norme dépasse rarement le seuil de 1,0 (hors début d'entraînement), le clipping est rarement actif, ce qui explique l'absence de différence significative. Il reste néanmoins une protection utile : le grand entraînement de la section 4 (`hidden_size` = 1024) subit des normes de gradient énormes (voir plus bas), que le clipping permet d'absorber.

3.5 Run 2 vs Run 3 : RNN vs LSTM

Le LSTM converge plus vite et plus loin : dès l'époque 2 son accuracy de validation (77,1 %) égale celle que le RNN atteint à la fin de l'entraînement, et il termine à 81,57 % (soit environ +4,3 points) avec une loss de validation de 0,571 contre 0,73. Sa norme de gradient est aussi environ deux fois plus faible (0,115 contre 0,27) et plus régulière. Le coût est un temps d'entraînement environ 1,75 fois plus long (73 s contre 42 s) et 3,5 fois plus de paramètres.

Explication mathématique. Pour un RNN d'Elman, $h_t = \tanh(W_{hh}h_{t-1} + W_{ih}x_t + b)$, d'où

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^{t} \operatorname{diag}\big(1 - h_j^2\big)\, W_{hh},$$

un produit de k matrices : il tend exponentiellement vers 0 (*vanishing*) ou vers l'infini (*exploding*) quand k croît, ce qui empêche de relier des événements éloignés (par exemple une répétition ou une barre de mesure ouverte 100 caractères plus tôt).

Le LSTM introduit un état de cellule mis à jour de façon *additive* :

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o),$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \qquad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t).$$

Le chemin direct du gradient à travers la cellule est

$$\frac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t) \quad (+ \text{ termes indirects via } h_{t-1}),$$

c'est-à-dire une simple multiplication terme à terme par la porte d'oubli, et non par une matrice pleine suivie d'une non-linéarité saturante. Lorsque $f_t \approx 1$, l'information et le gradient traversent de nombreux pas de temps presque sans atténuation, tandis que la porte d'entrée i_t décide de ce qui est écrit et la porte de sortie o_t de ce qui est lu. Le LSTM peut ainsi mémoriser la tonalité, la métrique ou la structure des répétitions sur de longues distances, ce que le RNN simple n'arrive qu'imparfaitement à faire.

3.6 Stabilité et sur-apprentissage

Les pertes de validation des trois modèles baissent globalement sur les 15 époques, avec de petites irrégularités pour les RNN (époques 5, 7, 11 et 14 selon les cas). Les pertes d'entraînement et de validation restent proches (par exemple 0,70 contre 0,73 pour le RNN sans clipping, 0,53 contre 0,57 pour le LSTM) : on n'observe pas de sur-apprentissage marqué sur 15 époques. L'écart train/validation légèrement croissant du LSTM suggère toutefois qu'il commencerait à sur-apprendre sur un entraînement plus long avec seulement 10 000 partitions, d'où l'intérêt de l'*Early Stopping* et d'un jeu plus grand à l'étape suivante. À noter : la loss d'entraînement est moyennée pendant l'époque alors que la validation est évaluée en fin d'époque, ce qui peut rendre la comparaison directe optimiste pour la validation.

3.7 Limite de la métrique d'accuracy (padding)

Une partition de 250 caractères est complétée par 250 espaces jusqu'à 500. Le modèle apprend très facilement que, après un espace de padding, le caractère suivant est un espace : ces prédictions sont quasi parfaites et comptent pour autant que les vraies notes dans l'accuracy. Comme la longueur moyenne (290) est bien inférieure à 500, au moins ≈ 42 % des positions sont en moyenne du padding. L'accuracy affichée est donc **artificiellement optimiste** par rapport à la capacité réelle du modèle à prédire des notes de musique.

La modification demandée consiste à ignorer l'indice du padding dans la perte :

```
pad_idx = char_to_index[' ']
loss_fn = nn.CrossEntropyLoss(ignore_index=pad_idx)
```

Les positions dont la cible vaut `pad_idx` ne contribuent alors ni à la perte ni aux gradients. Limite : l'espace est aussi un caractère légitime de la notation ABC (il sépare les groupes de notes) ; une solution plus propre consiste à utiliser un jeton de padding dédié, ou à construire un masque à partir de la longueur originale. Le même masque doit aussi être appliqué au calcul de l'accuracy.

4 Génération musicale, température et job Slurm

4.1 Efficacité algorithmique

À chaque itération, notre méthode ne transmet au modèle que le dernier caractère généré (tenseur de forme $(1, 1)$) avec l'état caché `hidden_state` conservé.

Sans conservation de l'état caché. À l'itération k, il faudrait retraiter toute la séquence de k caractères (en ignorant ici la longueur de l'amorce). Le nombre total de pas récurrents serait

$$1 + 2 + \cdots + N = \frac{N(N+1)}{2} \quad \Longrightarrow \quad \boxed{O(N^2)}.$$

Avec conservation de l'état caché. L'amorce est traitée une seule fois, puis chaque itération traite exactement un caractère :

$$\boxed{O(N)}.$$

Pour une architecture et des tailles de couches fixées, la conservation de `hidden_state` fait passer la complexité de quadratique à linéaire en N.

4.2 Rôle mathématique de la température

$$P(y_t = c) = \frac{\exp(z_{t,c}/T)}{\sum_{c'} \exp(z_{t,c'}/T)}.$$

— $T \to 0^+$: la distribution se concentre sur le logit maximal ; si ce maximum est unique, sa probabilité tend vers 1 et les autres vers 0. Cela correspond au **décodage glouton** (*greedy decoding*, `argmax`), déterministe et sujet aux boucles répétitives.

— $T \to +\infty$: les écarts entre logits deviennent négligeables devant T, et la distribution tend vers la loi uniforme $P(y_t = c) \to 1/V$ avec $V = 95$: le tirage devient quasi aléatoire.

$T = 1$ correspond à l'échantillonnage selon la distribution apprise par le modèle.

4.3 Écoute et analyse qualitative

Les trois générations utilisent le même modèle LSTM et la même amorce :

```
X:1
T:Irish Reel
M:4/4
L:1/8
K:D
```

Seule la température varie : $T = 0{,}2$, $T = 0{,}8$ et $T = 2{,}0$, avec 300 caractères générés.

Les trois partitions générées (fichiers `music_T02.abc`, `music_T08.abc` et `music_T20.abc`, également fournis dans le dossier `abc/`) sont reproduites ci-dessous. L'analyse de la validité syntaxique et de la durée des mesures est faite directement sur le texte (en 4/4 avec `L:1/8`, une mesure complète vaut 8 croches).

$T = 0{,}2$.

```
X:1
T:Irish Reel
M:4/4
L:1/8
K:D
 A2 | d2 d2 d2 c2 | B2 A2 A2 d2 | c2 B2 A2 A2 | B2 c2 d2 d2 | c2 B2 A2 G2 | F2 A2 A2 A2 |
 B2 c2 d2 c2 | B2 A2 A2 |]
```

— **Validité syntaxique** : entièrement valide. Une anacrouse (`A2`) puis 7 mesures complètes de 8 croches ; la dernière mesure est tronquée (6 croches) car la génération s'arrête à la longueur demandée.

— **Richesse musicale** : très pauvre. Seules des noires (`x2`) apparaissent, avec un mouvement quasi exclusivement conjoint dans un ambitus étroit (F→d) et des motifs qui se répètent presque à l'identique (`d2 d2 d2 c2`, `B2 A2 A2`…). Il n'y a ni reprise, ni variation rythmique, ni phrase identifiable : c'est le comportement quasi glouton, correct mais monotone.

$T = 0{,}8$.

```
X:1
T:Irish Reel
M:4/4
L:1/8
K:D
 A3 A F4 | A3 F E4 | A3 A F4 | F4 F4 | A3 A F3 F | F4 E4 | F3 A A4 | B A3 A4 | G4 F4 |
 G3 F F3 G | A3 G F3 F | G3 F E4 | F3 G A3 B | A G F2 E2 D2 | G4 F4 | G3 A B4 | A3 G F3 E |
 A4 D4 | G3 F E4 | F4 D4 | A3 F A3 A | A4 z2 |]
```

— **Validité syntaxique** : entièrement valide. 21 mesures complètes de 8 croches, puis une dernière mesure de 6 croches terminée par un silence (`z2`) et la barre finale `|]`.

— **Richesse musicale** : meilleure que pour $T = 0{,}2$. On trouve une cellule rythmique récurrente (3+1 croches suivies d'une note longue, comme `A3 A F4`) alternant avec des variations (`A G F2 E2 D2`, `F3 G A3 B`), un ambitus plus large (D→B) et une progression mélodique qui revient régulièrement sur A, F et E. La partition reste homogène (pas de reprise `|: :|`, rythme peu varié), mais elle est cohérente et évite l'enfermement dans une boucle.

$T = 2{,}0$.

```
X:1
T:Irish Reel
M:4/4
L:1/8
K:D
|: D2dG=FEF=D G2 :|[M:5/4]" Q11t C""A""F"H!Z[A-AA-c.[Ae-]2"^[x5d" g3"D" [fA]DE |] [[GEe]3 (-:!
    aId-D- !/ k |
[M:3/2] (ub/4 a4!te! MB2-)!f+!"A7NfC2BAiLserusb6?"[E] BGCB d]2/DE | [EAE]}[CEcF]1[BF] A[EA]Bc |
"Eb7" [BdE^DG]2-"Gb" [AA]3- BA(c][c-G]" F3)A, | [AA c=8: d{/e}8dc(dA[BB][AF]) [CE]3 (e'- [c'^
```

— **Validité syntaxique** : très dégradée. La première mesure contient 10 croches au lieu de 8 ; des symboles d'accords entre guillemets sont mal appariés (`" Q11t C"`, `"A"`, `"F"H`) ; des changements de métrique inattendus apparaissent (`[M:5/4]`, `[M:3/2]`) ; de nombreux caractères n'appartiennent pas à la grammaire des notes (`!Z`, `(ub/4`, `?`, `=8:`, `}`) ; des crochets et parenthèses restent ouverts, et le texte se termine au milieu d'un accord (`[c'^`). Aucune mesure ne respecte plus les 8 croches du 4/4.

— **Richesse musicale** : la diversité de caractères est maximale mais le résultat est chaotique et non jouable tel quel ; le modèle conserve néanmoins le vocabulaire ABC (barres, accords,

changements de mesure, décorations), ce qui montre qu'il reproduit le style local mais n'en respecte plus la structure globale.

Température	Syntaxe ABC	Diversité	Répétitions	Cohérence
$T = 0{,}2$	valide	faible	très nombreuses	monotone (uniquement des noires)
$T = 0{,}8$	valide	modérée	équilibrées	bonne, mesures toutes complètes
$T = 2{,}0$	invalide	très élevée	quasi nulles	chaotique

Table 5 – Synthèse qualitative des trois générations (amorce `Irish Reel`, 300 caractères).

Une température faible favorise les caractères les plus probables mais conduit à des répétitions ; une température élevée augmente la diversité au détriment de la validité syntaxique et musicale ; une température intermédiaire comme 0,8 offre en général le meilleur équilibre.

4.4 Entraînement à grande échelle via `sbatch`

Le script `run_music.sh` lance un entraînement non interactif du LSTM avec `hidden_size` = 1024, `subset_size` = 50 000, `max_grad_norm` = 1,0 et `num_epochs` = 30, puis génère les trois exemples musicaux à la fin de l'exécution. Le job a été soumis avec `sbatch` (job 49921) ; une seconde soumission (job 49933) restée en attente à cause de la limite `QOSMaxJobsPerUserLimit` a été annulée avec `scancel`.

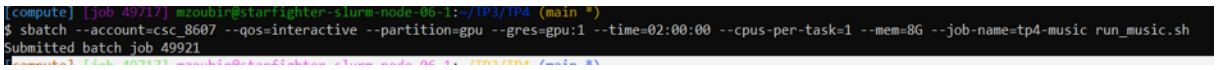


Figure 11 – Soumission du job Slurm.

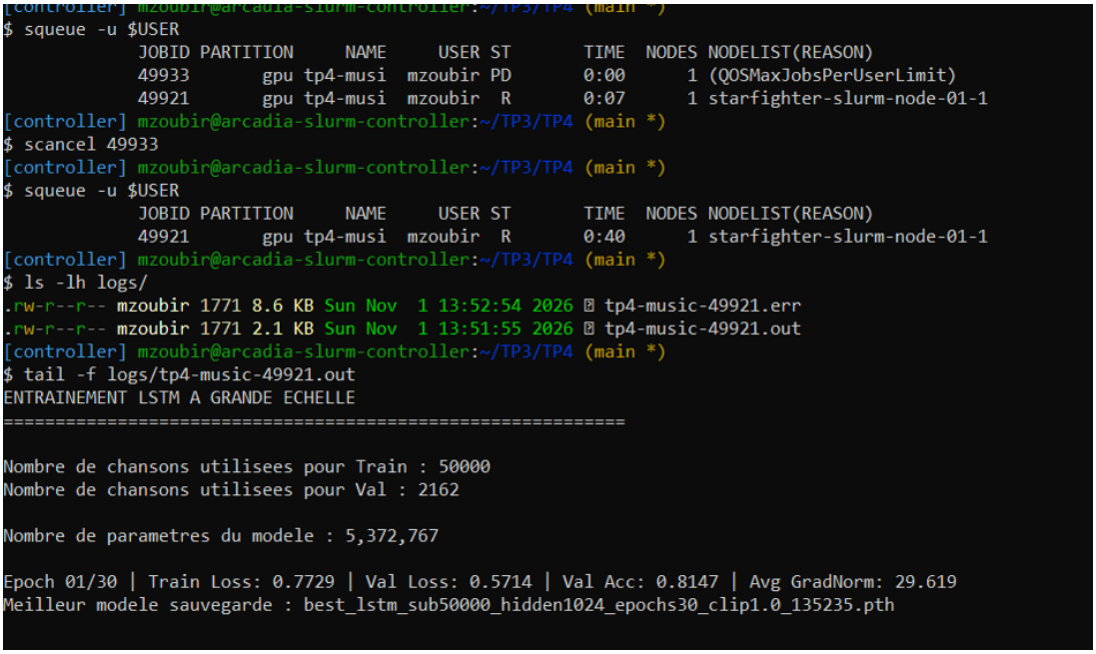


Figure 12 – État de la file Slurm et début du log d'entraînement du job 49921.

Le modèle comporte 5 372 767 paramètres, ce qui est cohérent avec le calcul théorique pour $H = 1024$:

$$\underbrace{95 \times 256}_{24\,320} + \underbrace{4(1024 \cdot 256 + 1024^2 + 2 \cdot 1024)}_{5\,251\,072} + \underbrace{1024 \cdot 95 + 95}_{97\,375} = 5\,372\,767.$$

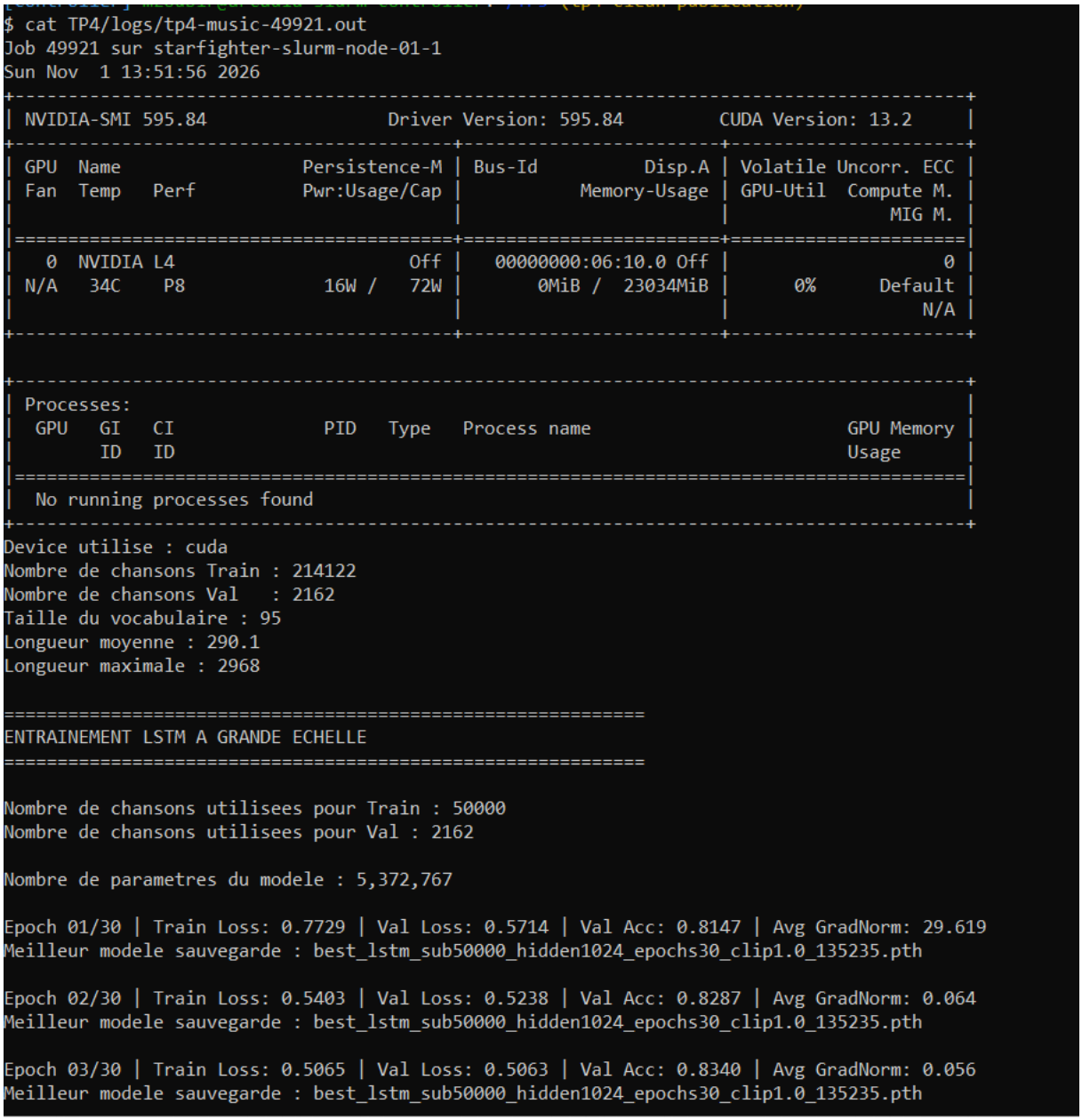


Figure 13 – Début du log du job 49921 (`logs/tp4-music-49921.out`).

Résultats. Le job s'est terminé avec succès (30 époques). Le meilleur modèle selon la loss de validation est celui de l'**époque 10** (loss de validation 0,4739, accuracy 84,61 %) ; c'est lui qui a été rechargé pour générer les trois partitions. Comparé au LSTM de la section 3 (10 000 partitions, $H = 512$, 15 époques : loss 0,5712, accuracy 81,57 %), le gain est d'environ $-0{,}10$ en loss et $+3{,}0$ points d'accuracy, ce qui reflète l'effet combiné de 5 fois plus de données et d'un modèle plus grand.

Époque	Loss train	Loss val.	Acc. val.	GradNorm moyenne
1	0,7729	0,5714	81,47 %	29,619
5	0,4722	0,4888	84,03 %	0,050
10	0,4625	**0,4739**	**84,61 %**	0,043
13	0,5750	0,5446	82,37 %	0,123
15	0,8214	0,7681	76,16 %	12 352,597
20	0,5761	0,5810	81,25 %	0,104
30	0,4981	0,5228	83,07 %	0,117

Table 6 – Quelques époques de l'entraînement à grande échelle (job 49921).

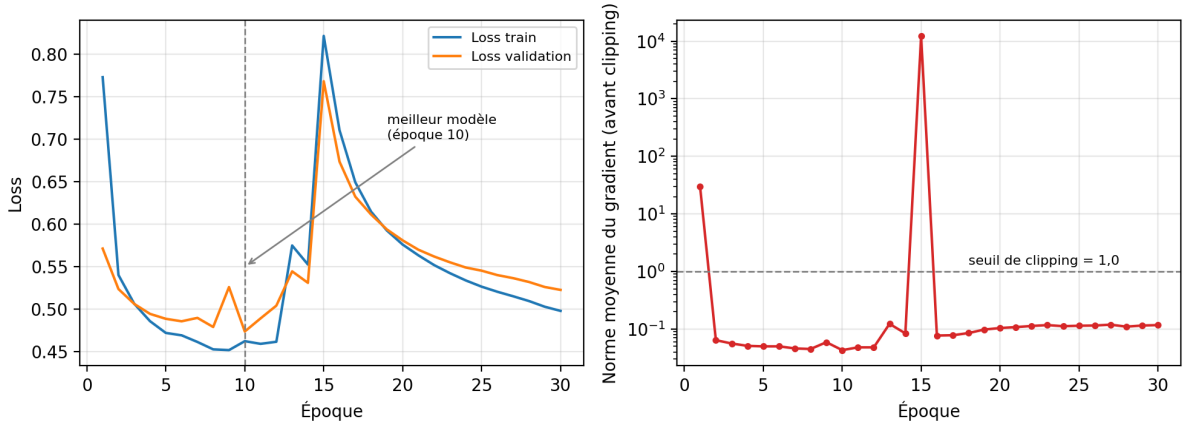


Figure 14 – Entraînement à grande échelle (tracé à partir du log). À gauche : loss d'entraînement et de validation ; à droite : norme moyenne du gradient avant clipping (échelle logarithmique).

Analyse.

— **Convergence initiale** : la loss de validation passe de 0,57 à 0,47 en 10 époques, sans écart notable entre loss d'entraînement et de validation (0,4625 contre 0,4739 à l'époque 10) : pas de sur-apprentissage, même avec 5,4 millions de paramètres.

— **Norme des gradients à l'époque 1** : la moyenne de 29,6 est très supérieure au seuil de clipping. Comme elle retombe à 0,06 dès l'époque 2, elle est dominée par quelques pas initiaux de norme très élevée ; sans clipping, ces mises à jour auraient été disproportionnées.

— **Instabilité des époques 13 à 15** : la loss remonte (0,5446 puis 0,7681 en validation) et la norme moyenne des gradients atteint $\approx 1{,}2 \times 10^4$ à l'époque 15 : c'est un épisode typique d'*exploding gradients*. Grâce au clipping à 1,0, chaque mise à jour reste de taille bornée : l'entraînement ne diverge pas (pas de `NaN`) et la loss de validation redescend progressivement jusqu'à 0,5228 à l'époque 30. Elle ne retrouve toutefois pas son minimum de l'époque 10. Une petite remontée isolée apparaît aussi à l'époque 9 (loss de validation 0,5261).

— **Rôle de la sauvegarde du meilleur modèle** : le dernier modèle (époque 30) est moins bon que celui de l'époque 10 (0,5228 contre 0,4739 en loss ; 83,07 % contre 84,61 %). Conserver le checkpoint de meilleure loss de validation et le recharger avant la génération permet donc de ne pas subir cette dégradation. Dans ce log, l'entraînement est allé jusqu'à l'époque 30 alors que la loss de validation ne s'améliore plus après l'époque 10 : l'Early Stopping n'a pas interrompu l'exécution.

Remarque sur l'instabilité. Ce pic suggère que le pas d'apprentissage $\mathrm{lr} = 5 \cdot 10^{-3}$ est élevé pour un LSTM de largeur 1024 avec Adam ; une décroissance du taux d'apprentissage (*scheduler*) ou une valeur plus faible serait une piste à tester, mais nous ne l'avons pas expérimentée ici.

Générations. Les trois partitions de la section précédente sont celles produites automatiquement à la fin de ce job, à partir du meilleur modèle (époque 10).